



PROYECTO 1: RAG SERVERLESS MULTINUBE

Raúl Rodrigo Pérez Marrero
Certified AI/LLM Solutions Architect
Módulo 2: Diseño de Infraestructura Escalable
Prof: Andrés Felipe Rojas Parra

Tabla de Contenidos

1. Definición del Caso de Uso del LLM.....	3
Indicadores Clave de Desempeño (KPIs).....	3
2. Selección del Modelo y Requerimientos de Infraestructura.....	3
Picos Esperados de Demanda.....	4
3. Selección y Justificación de los Patrones de Diseño LLM.....	4
Justificación Técnica.....	4
4. Contenerización con Docker.....	4
Estrategia de Cómputo.....	4
Dockerfile de Producción (Best Practices).....	4
5. Orquestación Serverless Multinube.....	6
Componentes por Proveedor.....	6
Lógica de Interconexión.....	6
6. Arquitectura Completa Multicloud.....	6
7. Diseño del Pipeline RAG Distribuido.....	7
Experimentos Obligatorios.....	7
8. Infraestructura de Serving de LLM (Managed).....	8
9. CI/CD Multinube con GitHub Actions.....	8
10. Optimización de Costos (FinOps Multicloud).....	8
Proyección de Línea Base (Estimado Mensual).....	8
11. Observabilidad y Métricas Cross-Cloud.....	8
12. Guía del Administrador.....	9
13. Guía de Usuario.....	13

1. Definición del Caso de Uso del LLM

En el actual ecosistema empresarial, la proliferación de silos de información en diversas nubes representa un desafío crítico para la implementación de Inteligencia Artificial Generativa. Las organizaciones requieren sistemas que no solo consulten datos, sino que lo hagan manteniendo la soberanía, seguridad y eficiencia operativa. La implementación de una plataforma **RAG (Retrieval-Augmented Generation)** permite orquestar bases de conocimiento privadas y distribuidas, proporcionando a los LLMs un contexto curado y en tiempo real sin los costos prohibitivos del ajuste fino (*fine-tuning*). He diseñado esta arquitectura **multi nube** bajo una premisa de resiliencia y optimización de capacidades: Azure para la orquestación y el hosting de la capa de presentación; AWS S3 para un almacenamiento de objetos altamente duradero y escalable; y GCP para la búsqueda vectorial de baja latencia mediante Vertex AI Search. Este enfoque no sólo mitiga el riesgo de *vendor lock-in* , sino que permite ejecutar cada etapa del pipeline RAG en el entorno donde es técnica y financieramente más eficiente. La realización de este proyecto está hecho pensando en un chatbot para una página web de laptops o PC's de cualquier marca, donde todos los documentos en la base de conocimientos deberán ser manuales de usuario y guías de configuración y soporte.

Indicadores Clave de Desempeño (KPIs)

Para validar la arquitectura y asegurar el éxito del despliegue, se establecen los siguientes umbrales técnicos:

- **Latencia Cross-Cloud:** Tiempo total del flujo (Azure → AWS → GCP → Azure) monitoreado para detectar cuellos de botella en la red.
- **Tokens por Segundo (Tokens/s):** Rendimiento de inferencia para asegurar una experiencia de usuario fluida.
- **Costo por 1k Tokens:** Métrica de eficiencia financiera para el control de presupuestos de IA.

2. Selección del Modelo y Requerimientos de Infraestructura

La plataforma soporta una configuración híbrida permitiendo alternar entre proveedores de inferencia según la criticidad y el costo.

Los componentes de arquitectura seleccionados son los siguientes:

- Lenguaje de Programación y Frameworks: Python y Flask para una aplicación monolítica de full stack (la página web es un html con fragmentos de CSS y Javascript).
- Hosteo Serverless de Aplicación y orquestación (frontend y backend): Azure Web App
- Servicio de Inferencia y Modelo de Lenguaje: Amazon Bedrock y Claude Sonnet 4.6
- Servicio de almacenamiento de documentos (base de conocimientos): Amazon S3
- Servicio de Embeddings y Modelo: Google Vertex AI y `text-multilingual-embedding-002`
- Servicio de Base de Datos Vectorial y Almacenamiento: Google Vector Search y Google Cloud Storage.

Picos Esperados de Demanda

Los picos de demanda para este caso de uso serán cuando las ventas de los productos ofrecidos por la empresa manufacturera suban, ya que se espera que muchos usuarios tengan

preguntas de cómo iniciar sus dispositivos por primera vez o cómo configurarlos. e.g. temporadas back-to-school, navidad, etc...

3. Selección y Justificación de los Patrones de Diseño LLM

Para este proyecto, se han implementado varios patrones de diseño LLM:

- **RAG:** Se utilizan los servicios de Google de Vector Search y Vector Store para proveer al LLM de contexto. Vector Search utiliza el algoritmo Tree-AH de BigQuery para las búsquedas semánticas.
- **Prompt Engineering:** implementado en el archivo `prompt_template.py`
 - Se implementa restricción de lenguaje.
 - Se implementa especificación de formato.
 - Se implementa el rol del modelo.
 - Se implementa la prevención de alucinaciones.
 - Se implementan medidas de seguridad para el usuario.
- **Grounded Generation Pattern:** Todos los fragmentos de la respuesta deben estar soportados por un documento o fuente.

Justificación Técnica

Mientras que el RAG básico depende exclusivamente de la similitud de cosenos en un espacio vectorial, los patrones de **prompt engineering** y **grounded generation**, aseguramos que el modelo tome un rol de soporte y que todas sus respuestas están basadas en los documentos de la fuente de conocimientos, evitando alucinaciones, además de ser respuestas seguras para el usuario, ya que se especifica que proporcione medidas de seguridad si los documentos las contienen.

4. Contenerización con Docker

Se implementan microservicios especializados para desacoplar la lógica de procesamiento de datos de la orquestación serverless.

Estrategia de Cómputo

Se utiliza Docker para los servicios de ingestión, chunking y validación, permitiendo portabilidad absoluta entre nubes si fuera necesario migrar de Azure Web Apps a Cloud Run.

Trivy está implementado en el pipeline de github actions.

Dockerfile de Producción (Best Practices)

```
# Stage 1: Imagen base con Python 3.13 para construcción.
FROM python:3.13-slim AS builder

# Mantiene el contenedor actualizado y define variables de entorno
ENV DEBIAN_FRONTEND=noninteractive
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
ENV APP_HOME=/workspace/app

# Crear directorios de trabajo
WORKDIR ${APP_HOME}
```

```

# Instalar dependencias del sistema necesarias para OpenCV y PDF
parsing
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    cmake \
    libglib2.0-0 \
    libsm6 \
    libxext6 \
    libxrender-dev \
    libpoppler-cpp-dev \
    poppler-utils \
    && rm -rf /var/lib/apt/lists/* \
    && pip install --no-cache-dir --upgrade pip

# Crear un ambiente virtual
RUN python -m venv /opt/venv

# Forzar al contenedor a usar los binarios del ambiente virtual
automaticamente.
ENV PATH="/opt/venv/bin:$PATH"

# Copiar archivo de requerimientos del proyecto
COPY requirements.txt ${APP_HOME}

#Instalar librerias adicionales necesitadas por el proyecto
RUN pip install --no-cache-dir -r requirements.txt

# --- Stage 2: Imagen minima final de runtime ---
FROM python:3.13-slim AS runner

ENV DEBIAN_FRONTEND=noninteractive
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
ENV APP_HOME=/workspace/app
ENV PATH="/opt/venv/bin:$PATH"

# Crear directorios de trabajo
WORKDIR ${APP_HOME}

# Crear grupo y usuario no root. y cambiar a ese usuario
RUN groupadd --gid 10001 appgroup && \
    useradd --uid 10001 --gid appgroup --shell /bin/false
--create-home appuser

# Copiar el ambiente virtual de la imagen constructora.
COPY --from=builder /opt/venv /opt/venv

```

```
# Copiar archivos del proyecto
COPY --chown=appuser:appgroup . ${APP_HOME}

USER appuser
# Exponer puerto 8000 para Flask
EXPOSE 8000

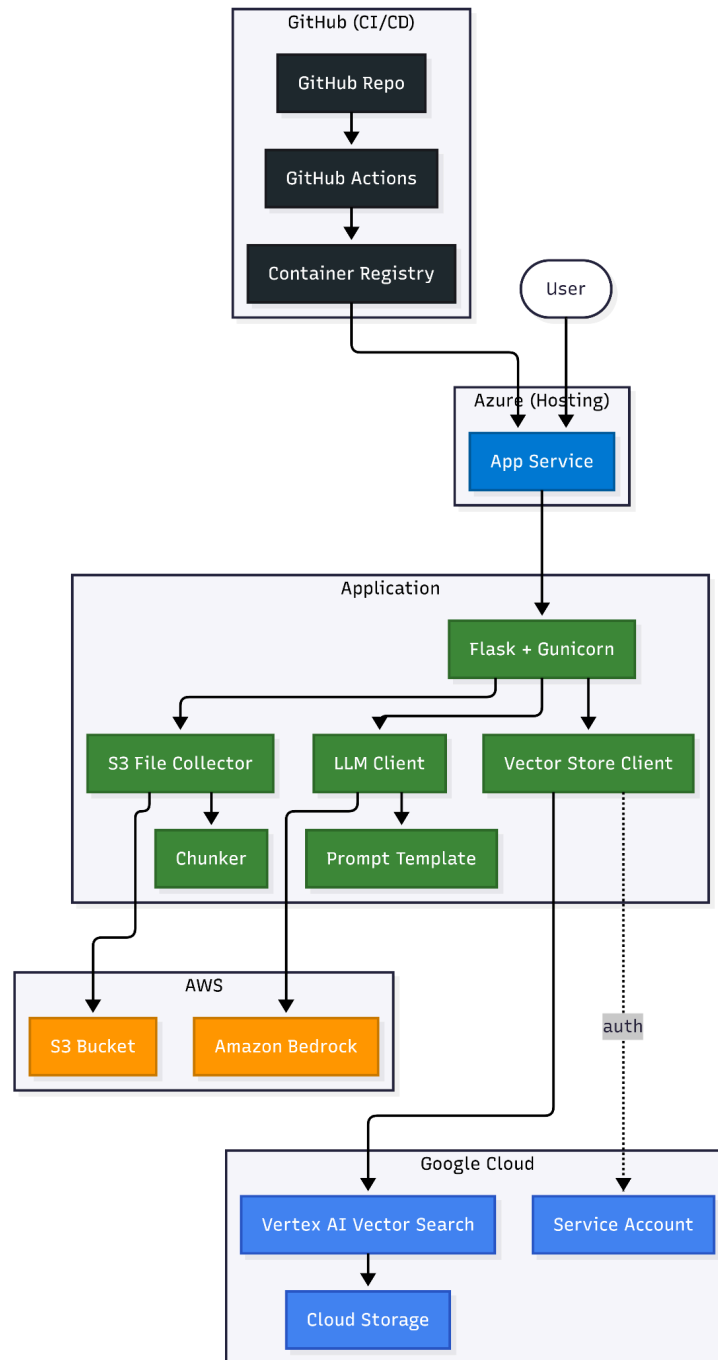
# Comando de ejecución
CMD ["gunicorn", "--preload", "app:app"]
```

5. Orquestación Serverless Multinube

La arquitectura no usa servicio de Kubernetes para no agregar complejidad innecesaria al sistema. En cambio usa una aplicación contenerizada y varios servicios de diferentes proveedores de nube, tomando en cuenta las bondades de cada proveedor, como performance, costos y utilidad de cada servicio.

Componentes por Proveedor

- **Azure:**
 - **Azure Web App:** Proporciona el ambiente para alojar la aplicación monolítica. (Tanto el backend u orquestador como el frontend)
- **AWS:**
 - **Amazon S3:** Un servicio de almacenamiento en la nube, utilizado para guardar los documentos que los usuarios puedan subir.
 - **Amazon Bedrock:** Proporciona acceso mediante API única a modelos fundacionales líderes de IA generativa.
- **GCP:**
 - **Service Account:** para acceder a los servicios de GCP.
 - **Vector Search:** Para recuperar los vectores o chunks matemáticamente más relevantes a la petición del usuario desde la base de datos vectorial.
 - **Vertex AI:** Para la generación de embeddings.
 - **Google Cloud Storage:** para guardar los archivos o chunks generados por los embeddings.
- **Github:**
 - **Actions:** Servicios de CI/CD para nuestro repositorio.



Lógica de Interconexión

La comunicación inter-nube se realiza mediante **Service Accounts** y **Access Tokens**. Se mitigan las vulnerabilidades no guardando directamente las credenciales en el repo, si no que se manejan mediante variables de ambiente.

6. Arquitectura Completa Multicloud

Hay dos principales flujos de datos, uno de carga de documentos y otro de conversación con el chatbot o modelo.

Flujo de carga:

1. **Frontend (Azure Web App):** Aplicación recibe la petición de carga de archivos.
2. **Almacenamiento de Documentos (Amazon S3):** El bucket de almacenamiento de S3 almacena los archivos en la nube.
3. **Chunking (Azure Web App):** La aplicación recibe el estatus de la carga de archivos y se encarga de elaborar las particiones de los archivos cargados exitosamente.
4. **Generación de Embeddings (Google Vertex AI):** El servicio de Google Vertex AI se encarga de generar los embeddings mediante el modelo especificado desde la llamada de la API.
5. **Guardado de Base de Datos Vectorial (Google Cloud Storage):** Los resultados de la generación de embeddings se guardan en un bucket de Google Cloud Storage.

Flujo de conversación:

1. **Frontend (Azure Web App):** Aplicación recibe la consulta.
2. **Búsqueda en base de datos Vectorial (Google Vector Search):** Se realiza el *Retrieval* en Vector Search y se regresan los chunks más relevantes al orquestador.
3. **Orquestador (Azure Web App):** El orquestador (backend) recibe los chunks y genera la consulta con el contexto con un prompt de sistema.
4. **Inferencia (AWS Bedrock):** Se envía la consulta a Bedrock y se genera una respuesta
5. **Frontend (Azure Web App):** El Frontend muestra la respuesta recibida.

7. Diseño del Pipeline RAG Distribuido

El pipeline se divide en 7 etapas para maximizar la escalabilidad:

1. **Ingesta (Amazon S3):** Carga masiva en S3.
2. **Chunking (Azure Web App):** Segmentación inteligente. **Nota Arquitectónica:** Se utiliza el orquestador (o backend) para simplificar el despliegue.
3. **Embeddings (Google Vertex AI):** Generación de vectores (ej. `text-multilingual-embedding-002`).
4. **Vector Store (Vertex AI Search):** Almacenamiento e indexación.
5. **Retrieval (Vector Search):** Búsqueda de los Top-K fragmentos.
6. **LLM Inference (Amazon Bedrock):** Generación de respuesta final.
7. **Frontend (Azure Web App):** Despliegue de resultados al usuario e interfaz.

Experimentos Obligatorios

Los experimentos que se hicieron fueron primeramente con chunks muy grandes, de 6000 tokens pero, descubrí que se estaba mandando demasiado contexto, ya que mandaba aproximadamente 100 chunks por consulta, esto ocasiona que el LLM tuviera mucho contexto “basura”, el cual afecta en costos y en la calidad de la respuesta.

Después de esto, intenté generar chunks de 2000 tokens, pero igual, era demasiado y se mandaba mucho contexto basura.

Después de esto, investigué cuál era el tamaño óptimo para mi caso de uso, y descubrí que los chunks deberían tener un tamaño de 500 tokens, lo configure acorde y ya empecé a generar los chunks y obtener respuestas de calidad, mandando 50 chunks por consulta.

El traslape siempre lo mantuve en 10%, en los tres experimentos, ya que también coincidía con lo que decía la investigación.

8. Infraestructura de Serving de LLM (Managed)

Para garantizar la privacidad, se utilizó Amazon Bedrock con el modelo Claude Sonnet 4.5

9. CI/CD Multinube con GitHub Actions

El despliegue está totalmente automatizado mediante **GitHub Actions**.

- **Build & Push:** Las imágenes Docker se suben a **GitHub Container Registry (ghcr.io)**.
- **Deploy:** GitHub Actions actualiza solamente Azure Web App, con toda la configuración contenida en el código del orquestador y las variables de ambiente.

10. Optimización de Costos (FinOps Multicloud)

Proyección de Línea Base (Estimado Mensual)

Categoría	Costo Estimado (Baseline)	Detalle de Optimización
Inferencia LLM (Amazon Bedrock)	\$60.00	"Uso de modelos ""Mini"" para tareas simples."
Vector Search (GCP)	\$50.00	Tier "Standard" con autoescalado.
Storage & Egress (GCP Cloud Storage)	\$25.00	Compresión de embeddings y caching.
Serverless Ops	\$15.00	Uso de capas gratuitas (F1/Free Tier).
Total Estimado	\$150.00	Costo por Request: ~\$0.008

11. Observabilidad y Métricas Cross-Cloud

- **Monitoreo Centralizado:** Integración de logs para la telemetría de la aplicación utilizando Azure Log Streams.
- **Métricas por pregunta:** El sitio tiene la función de desplegar métricas por cada consulta realizada, tales como (seguido de cada métrica, viene un ejemplo real de una pregunta realizada al LLM):
 - **Tiempo de retrieval(GCP):** 13.17s
 - **Tiempo de LLM(AWS):** 7.52s
 - **Tiempo total:** 20.68s
 - **Input tokens:** 10,248
 - **Output tokens:** 434
 - **Costo input:** \$0.0307 USD
 - **Costo output:** \$0.0065 USD
 - **Costo total:** \$0.0373 USD

12. Guía del Administrador

Primeros pasos

- Verifica tu instalación de Python con `python3 --version` para Mac o, para Windows: `python --version`.
 - Si no está instalado, debes instalar [Python](#)
- Verifica que pip esté instalado con `python3 -m pip --version` para Mac o, para Windows: `python -m pip --version`.
- Instala las extensiones "Python" y "Python Debugger" en el IDE de tu preferencia.

Luego, ejecuta el siguiente script en tu terminal:

Para macOS/linux:

```
python3 -m venv .venv # crear un ambiente virtual para tu proyecto
source .venv/bin/activate # activar el ambiente virtual en tu
proyecto
```

Para windows:

```
python -m venv .venv # crear un ambiente virtual para tu proyecto
.venv\Scripts\activate # activar el ambiente virtual en tu proyecto
```

Luego instala las dependencias de Python con:

```
pip install -r requirements.txt
```

Pasos para el despliegue

AWS:

1. Crea tu cuenta en [AWS](https://aws.amazon.com/free/)
2. Crea tu clave de acceso. Esto nos ayudará con la interconexión de proveedores de nube (Azure a AWS)
 - a. Ve al menú en la esquina superior derecha de la pantalla de tu consola de AWS
 - b. Haz clic en credenciales de seguridad (security credentials)
 - c. Ve a la sección de Claves de Acceso (Access Keys)
 - d. Haz clic en Crear Clave de Acceso (Create Access Key)
 - e. Copia todos los valores en un bloc de notas, editor de texto o archivo de variables de entorno (sigue el ejemplo en el archivo `.env.sample`)
3. Crea un bucket de S3. Esto nos ayudará a almacenar nuestra base de conocimientos
 - a. Configura un espacio de nombres (namespace) para el bucket
 - b. Desactiva las ACLs
 - c. Bloquea el acceso público (public access)
 - d. Desactiva el control de versiones del bucket (bucket versioning)
 - e. Selecciona el cifrado predeterminado (default encryption)

- f. Desactiva el bloqueo de objetos (Object lock)
4. Para el modelo de Bedrock, debes enviar un caso de uso para el primer uso. (Si ya has utilizado modelos de Anthropic en tu AWS, no es necesario seguir estos pasos)
 - a. Ve a "Amazon Bedrock"
 - b. Busca la opción "Catálogo de modelos" (Model catalog) en el panel izquierdo
 - c. Selecciona cualquier modelo de Anthropic
 - d. Aparecerá una notificación en la parte superior pidiéndote que completes un caso de uso. Haz clic allí y completa el formulario del caso de uso

Github:

1. Crea tu cuenta en [Github](https://github.com).
2. Crea un Token de Acceso Personal (PAT) (Clásico) en GitHub
 - a. Clickea en tu icono de perfil en la esquina superior derecha
 - b. Clickea en la opción de Configuración (Settings)
 - c. Clickea en la opción de Configuración de Desarrolladores (Developer Settings) hasta abajo en el menú de la izquierda
 - d. Clickea en la opción de Tokens de Acceso Personal (Personal Access Tokens) en el menú de la izquierda
 - e. Clickea en Token clásica (Tokens (classic))
 - f. Clickea en el botón de "Generar token nueva" (Generate new token) en la esquina superior derecha
 - g. Clickea en la opción de "Generar nueva token clásica" (Generate new token (classic)).
 - h. Dale un nombre a tu token.
 - i. Marca los permisos de `repo` y `read:packages`.
 - j. Clickea en el botón de "Generar token" (Generate token) hasta abajo.
 - k. Copia la Token personal de acceso en el editor de texto de tu preferencia

Azure:

1. Crea tu cuenta de [Azure](https://portal.azure.com).
2. Crea una Suscripción (nómbrela como deseas)
3. Crea un Grupo de Recursos (Resource Group) dentro de la Suscripción creada en el paso 2 (nómbrela como deseas)
 - a. Selecciona la región que te resulte más conveniente (por ejemplo, south central US)
4. Crea un App Service
 - a. Selecciona simplemente "Web App" cuando hagas clic en el botón de crear
 - i. Selecciona la Suscripción y el Grupo de Recursos creados en los pasos 2 y 3 respectivamente
 - ii. Nombra tu webapp como deseas
 - iii. Selecciona la opción "Contenedor" (Container) en el campo Publicar (botón de radio)
 - iv. Selecciona el sistema operativo; por defecto y convenientemente es Linux
 - v. Selecciona la región más conveniente para ti (por ejemplo, south central US)
 - vi. Selecciona el plan de Linux, o crea uno nuevo si no tienes uno
 - vii. Selecciona el plan de precios (Para este proyecto seleccionará "Free F1")
 - b. Haz clic en "Siguiente: Base de datos >"

- i. NO crees una base de datos
- c. Haz clic en "Siguiendo: Contenedor >"
 - i. Desactiva el soporte de Sidecar
 - ii. Selecciona la fuente de la imagen: otros registros de contenedores (other container registries)
 - iii. Tipo de acceso: Privado
 - iv. URL del servidor de registro: <https://ghcr.io>
 - v. Nombre de usuario: <tu_nombre_de_usuario_de_github>
 - vi. Contraseña: <pat_creado_en_la_seccion_de_github>
 - vii. Imagen y etiqueta: <nombre_de_tu_contenedor_docker>:<version>
 - viii. Comando de inicio: `unicorn app:app`
- d. Haz clic en "Siguiendo: Redes >"
 - i. Habilita el acceso público
- e. Haz clic en "Siguiendo: Monitoreo + seguridad >"
- f. Haz clic en "Siguiendo: Etiquetas >" (Opcionalmente, añade las etiquetas que desees)
- g. Haz clic en "Siguiendo: Revisar + crear >"
 - i. Revisa la configuración que ingresaste y finalmente haz clic en "Crear"
- h. Ve a tu aplicación web recién creada desde tu portal de inicio
 - i. Abre el nodo del árbol de Configuración (Settings) a la izquierda
 - ii. Haz clic en Configuración (Configuration)
 - iii. Asegúrate de que las opciones "SCM Basic Auth Publishing Credentials" y "FTP Basic Auth Publishing Credentials" estén habilitadas (las dos primeras opciones del panel)
 - iv. Haz clic en el botón "Aplicar" (Apply) en la parte inferior
 - v. Haz clic en la opción "Información general" (Overview) a la izquierda
 - vi. Haz clic en "Descargar perfil de publicación" (Download publish profile) desde la pantalla de Información general en la parte superior
 - vii. Guarda el archivo donde prefieras
- i. Configurar variables de entorno
 - i. Busca y haz clic en la opción "Configuración" (Settings) en el panel izquierdo
 - ii. Busca y haz clic en la opción "Variables de entorno" (Environment variables) debajo
 - iii. Asegúrate de configurar las variables de entorno detalladas en el archivo `.env.sample`

Github & Github Actions:

1. Ve a la configuración de tu repositorio (Repository Settings)
 - a. Crea los Secretos necesarios
 - i. Busca y haz clic en la opción "Variables y Secretos" (Secrets and Variables) a la izquierda.
 - ii. Haz clic en la subopción "Actions".
 - iii. Haz clic en el botón "Nuevo secreto de repositorio" (New repository secret).
 - iv. El nuevo secreto debe nombrarse `AZURE_WEBAPP_PUBLISH_PROFILE`.
 - v. Copia el contenido del archivo del perfil de publicación de la última sección y haz clic en "Add secret".

- vi. Ahora, cada vez que realices un push a la rama principal (o predeterminada), tu contenedor se construirá y se desplegará en tu aplicación web de Azure.
- b. Otorga permisos al flujo de trabajo de Actions
 - i. Busca y haz clic en la opción "Actions" en el lado izquierdo de la página de configuración
 - ii. Haz clic en la subopción "General"
 - iii. Busca la sección "Workflow permissions"
 - iv. Habilita la opción "Read and write permissions"
 - v. Haz clic en "Guardar" (Save)

Google Cloud:

1. Abre Google Cloud Platform y crea una cuenta
2. En la esquina superior izquierda, verás el botón del proyecto; haz clic en él y crea un proyecto
3. Claves de API y cuenta de servicio
 - a. En la barra de búsqueda, escribe Cuenta de Servicio (Service Account) y ve allí.
 - b. En la parte superior de la página, hay un botón llamado "Crear cuenta de servicio". Haz clic en él.
 - c. Dale un nombre a tu cuenta de servicio; se asignará automáticamente un ID basado en el nombre dado.
 - d. Haz clic en el botón "Crear y cerrar" en la parte inferior.
 - e. Añadir clave.
 - i. Identifica la fila de tu cuenta de servicio recién creada y haz clic en los 3 puntos a la derecha de la tabla.
 - ii. Haz clic en la opción "Administrar claves".
 - iii. Haz clic en "Añadir clave" en la parte inferior y selecciona "Crear nueva clave".
 - iv. Selecciona "JSON" en la ventana emergente y haz clic en "Crear". Guarda el archivo en la ubicación de tu preferencia.
 - vi. Codifica el contenido del archivo en formato base64. Copia el valor codificado en la variable de entorno `GOOGLE_SA_CREDENTIALS_BASE64` (Asegúrate de usar un codificador seguro y que no sea en línea).
 - f. Añadir Roles
 - i. Haz clic en la pestaña "Permisos" (Permissions)
 - ii. Haz clic en el botón "Administrar acceso" (Manage Access)
 - iii. Busca el rol "Storage Object User" y selecciónalo
 - iv. Haz clic en el botón "Añadir otro rol"
 - v. Busca el rol "Vector Search Service Agent" y selecciónalo
 - vi. Haz clic en el botón "Añadir otro rol"
 - vii. Busca el rol "Vertex AI RAG Data Service Agent" y selecciónalo
 - viii. Haz clic en Guardar
4. Google Cloud Storage (GCS)
 - a. En la barra de búsqueda, escribe GCS y ve allí
 - b. Crea un nuevo bucket y dale un nombre. Haz clic en "Continuar"
 - c. Selecciona región única y elige una región (preferiblemente que tenga una baja emisión de CO2 y esté cerca de las regiones seleccionadas en Azure/AWS). Haz clic en continuar

- d. Habilita el espacio de nombres jerárquicos. Haz clic en Continuar
 - e. Desactiva el acceso público y el Control de Acceso Uniforme. Haz clic en continuar
 - f. Desactiva la política de eliminación suave (Soft delete), el control de versiones de objetos y la política de retención del bucket. Haz clic en Crear
5. Búsqueda de Vectores (Vector Search)
- a. En la barra de búsqueda, escribe Vector Search y ve allí
 - b. Si no las tienes habilitadas, aparecerá una notificación. Haz clic en Habilitar APIs y activa las APIs marcadas como desactivadas
 - c. Selecciona la misma región que seleccionaste en el paso 4.3
 - d. Crea un Índice
 - i. Haz clic en "Crear nuevo índice". Completa los campos requeridos
 - ii. Selecciona una carpeta de GCS
 - iii. En el campo de dimensiones, escribe 768 (Esto aplica para el modelo seleccionado)
 - iv. En el campo "Approximate neighbors count", escribe 50 (parámetro top-k)
 - v. En el campo "Update method", selecciona Stream
 - vi. Haz clic en el botón "Crear" en la parte inferior
 - e. Crea endpoints de índice
 - i. Haz clic en la pestaña superior que dice "Index endpoints"
 - ii. Haz clic en "Create new endpoint"
 - iii. Escribe un nombre para tu endpoint
 - iv. Selecciona "Standard"
 - v. Haz clic en "Crear"
 - f. Desplegar índice
 - i. Ve a la pestaña "Indexes" de nuevo
 - ii. En la fila donde aparece tu índice recién creado en el paso 4, verás el botón "Deploy"; haz clic allí
 - iii. Escribe un nombre para tu despliegue
 - iv. Selecciona el endpoint de índice creado en el paso 5
 - v. Elige el tipo de máquina Standard
 - vi. Habilita el autoescalado
 - vii. En el número mínimo de réplicas, escribe 1
 - viii. En el número máximo de réplicas, escribe 5
 - ix. Haz clic en "Deploy". (Este paso tardará más de 15 minutos)
 - g. Llena los secretos faltantes. Asegúrate de llenar todos los secretos que empiecen con `GOOGLE` o `GCP` para empezar a usar estos servicios.
 - i. Vete a la web app creada en azure y ve a la sección de variables de ambiente.
 - ii. Copia cada valor de tu archivo `.env.sample` al portal de azure.
6. Listo! ahora tienes todo lo necesario para el despliegue de este proyecto. Asegúrate de confirmar tus cambios en la rama predeterminada de tu repositorio para que puedas ver los cambios en vivo en internet.

13. Guia de Usuario

1. Ir a la página web:
<https://backend-rag-ecdjauaqfsabhke7.southcentralus-01.azurewebsites.net/>
2. Para la revisión/modificación de la base de documentos:
 - a. Hacer click en la flecha a la derecha de la sección “Archivos en bucket”
 - b. A continuación se desplegará una lista de todos los archivos disponibles en la base de conocimientos del chatbot.
 - c. Por cada archivo, encontrarás un botón de “Descargar” y “Eliminar”, puedes interactuar con ellos para analizar o modificar la base de documentos.
 - i. Si das click en “Eliminar”, una vez que hayas terminado de eliminar todos los archivos pertinentes, da click en el botón “Recargar cache”, este proceso puede tardar un poco, es para que los cambios se apliquen a la base de conocimientos.
3. Para cargar archivos:
 - a. Selecciona el botón “Elegir Archivos”.
 - b. Selecciona el/los archivo(s) que desees subir.
 - c. Click en Subir.
 - d. Repite el proceso tantas veces sea necesario.
 - e. El sistema solo admite archivos PDF de soporte de productos electrónicos, si subes otro tipo de PDF 's, es probable que el modelo responda cosas incoherentes.
 - f. Una vez hayas terminado de subir todos los archivos, da click en el botón “Recargar cache”, este proceso puede tardar un poco, pero es para que tus archivos se tomen en cuenta.
4. Para hacer una pregunta, solo escribe tu pregunta en el cuadro de texto y da click en preguntar, el modelo hará lo posible por responder tu pregunta basado en los archivos de la base de conocimientos.